

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине «Проектирование и реализация баз данных»

Тема: Запросы на выборку и модификацию данных.
Представления. Работа с индексами.

Выполнил	Мезенцев Б.Г.; К 3239.
Проверил	Говорова М.М.
Дата	13.04.2026

Санкт-Петербург 2026

1 Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2 Практическое задание

- Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
- Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
- Изучить графическое представление запросов и просмотреть историю запросов.
- Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

3 Схема базы данных

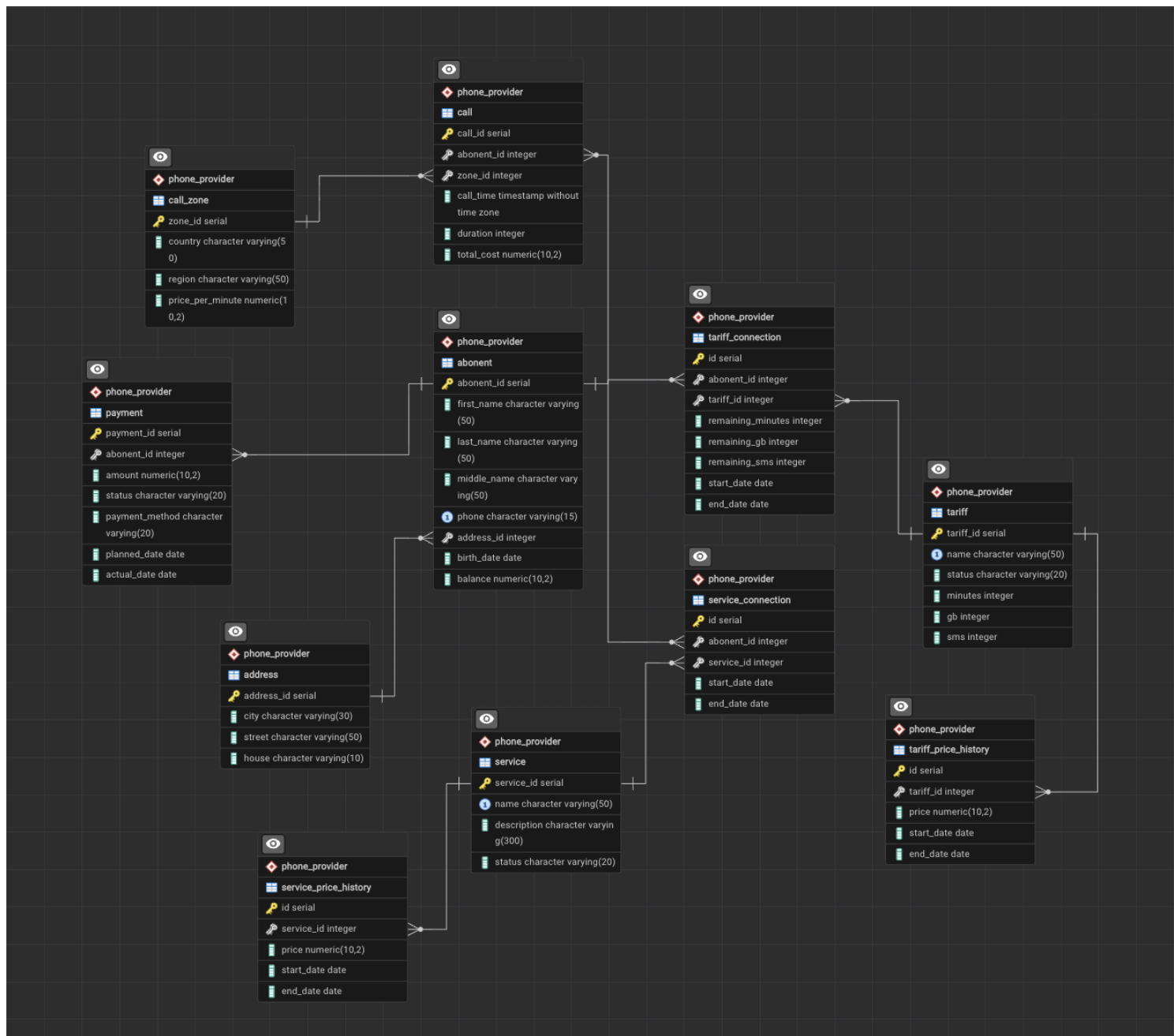


Рис. 1 — ERD-диаграмма базы данных из ЛР3

4 Выполнение лабораторной работы №4

4.1 Запросы к базе данных

Запрос 1. Вывести суммарное время переговоров каждого абонента за заданный период.

```
1 SELECT
2     a.abonent_id,
3     a.first_name,
4     a.last_name,
5     SUM(c.duration) AS total_duration
6 FROM phone_provider.call c
7 JOIN phone_provider.abonent a ON c.abonent_id = a.abonent_id
8 WHERE c.call_time BETWEEN '2026-03-01' AND '2026-03-31'
9 GROUP BY a.abonent_id, a.first_name, a.last_name;
```

Data Output Messages Notifications

Showing rows: 1 to 2

	abonent_id [PK] integer	first_name character varying (50)	last_name character varying (50)	total_duration bigint
1	1	Иван	Иванов	15
2	2	Петр	Петров	3

Запрос 2. Найти среднюю продолжительность разговора абонента АТС.

Query Query History AI Assistant

```
1 -- Средняя продолжительность разговора абонента
2 SELECT AVG(duration) AS avg_duration
3 FROM phone_provider.call;
```

Data Output Messages Notifications

	avg_duration numeric
1	6.0000000000000000

Запрос 3. Вывести количество междугородных переговоров каждого абонента. В данном запросе считаем звонки междугородными если название города не Москва.

Query

Query History

AI Assistant

1

-- Количество междугородных переговоров каждого абонента

2

SELECT

3

a.abonent_id,

4

a.first_name,

5

COUNT(c.call_id) AS intercity_calls

6

FROM phone_provider.call c

7

JOIN phone_provider.abonent a ON c.abonent_id = a.abonent_id

8

JOIN phone_provider.call_zone z ON c.zone_id = z.zone_id

9

WHERE z.region != 'Москва'

10

GROUP BY a.abonent_id, a.first_name;

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

⤴️

SQL

	abonent_id [PK] integer	first_name character varying (50)	intercity_calls bigint
1	2	Петр	1
2	1	Иван	1

Запрос 4. Вывести список абонентов, продолжительность разговоров которых за прошедшую неделю была максимальной.

Query	Query History	AI Assistant
1		-- Список абонентов, продолжительность разговоров которых за
2		-- прошедшую неделю была максимальной.
3		SELECT *
4		FROM (
5		SELECT
6		a.abonent_id,
7		a.first_name,
8		SUM(c.duration) AS total_duration
9		FROM phone_provider.call c
10		JOIN phone_provider.abonent a ON c.abonent_id = a.abonent_id
11		WHERE c.call_time >= CURRENT_DATE - INTERVAL '7 days'
12		GROUP BY a.abonent_id, a.first_name
13		) t
14		WHERE total_duration = (
15		SELECT MAX(total_duration)
16		FROM (
17		SELECT SUM(duration) AS total_duration
18		FROM phone_provider.call
19		WHERE call_time >= CURRENT_DATE - INTERVAL '7 days'
20		GROUP BY abonent_id
21		) sub
22		);
23		

Запрос 5. Сколько звонков было сделано в каждый из городов России.

Query Query History AI Assistant		
1	-- Сколько звонков было сделано в каждый из городов России	
2	SELECT	
3	z.region,	
4	COUNT(c.call_id) AS call_count	
5	FROM phone_provider.call c	
6	JOIN phone_provider.call_zone z ON c.zone_id = z.zone_id	
7	WHERE z.country = 'Россия'	
8	GROUP BY z.region;	

Data Output Messages Notifications		
SQL		
	region character varying (50)	call_count bigint
1	Москва	1
2	Регионы	1

Запрос 6. Вывести список абонентов, звонивших только в ночное время.

Query Query History AI Assistant		
1	-- Список абонентов, звонивших только в ночное время	
2	SELECT a.abonent_id, a.first_name	
3	FROM phone_provider.abonent a	
4	WHERE EXISTS (	
5	SELECT 1	
6	FROM phone_provider.call c	
7	WHERE c.abonent_id = a.abonent_id	
8	)	
9	AND NOT EXISTS (	
10	SELECT 1	
11	FROM phone_provider.call c	
12	WHERE c.abonent_id = a.abonent_id	
13	AND EXTRACT(HOUR FROM c.call_time) NOT BETWEEN 0 AND 6	
14	);	

Data Output Messages Notifications		
SQL		
	abonent_id [PK] integer	first_name character varying (50)

Запрос 7. Вывести список абонентов, время разговоров которых превышает среднее для этой же зоны.

Query Query History AI Assistant

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

-- Список абонентов, время разговоров которых превышает среднее для этой же зоны
SELECT
 a.abonent_id,
 a.first_name,
 c.duration,
 z.zone_id
FROM phone_provider.call c
JOIN phone_provider.abonent a ON c.abonent_id = a.abonent_id
JOIN phone_provider.call_zone z ON c.zone_id = z.zone_id
WHERE c.duration > (
 SELECT AVG(c2.duration)
 FROM phone_provider.call c2
 WHERE c2.zone_id = c.zone_id
);

Data Output Messages Notifications



abonent_id	first_name	duration	zone_id
integer	character varying (50)	integer	integer

4.2 Создание представлений

Представление 1. Было создано представление, содержащее сведения обо всех абонентах и их переговорах за прошедший месяц:

```
1 CREATE OR REPLACE VIEW phone_provider.abonent_calls_last_month AS
2 SELECT
3     a.abonent_id,
4     a.first_name,
5     a.last_name,
6     c.call_id,
7     c.call_time,
8     c.duration,
9     c.total_cost,
10    z.country,
11    z.region
12 FROM phone_provider.abonent a
13 JOIN phone_provider.call c ON a.abonent_id = c.abonent_id
14 JOIN phone_provider.call_zone z ON c.zone_id = z.zone_id
15 WHERE c.call_time >= CURRENT_DATE - INTERVAL '1 month';
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 124 msec.

Просмотр содержимого представления abonent_calls_last_month:

Query

Query History

AI Assistant

Scratch Pad

1

SELECT * FROM phone_provider.abonent_calls_last_month

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

Page No: 1 of 1

	abonent_id integer	first_name character varying (50)	last_name character varying (50)	call_id integer	call_time timestamp without time zone	duration integer	total_cost numeric (10,2)	country character varying (50)	region character varying (50)
1	1	Иван	Иванов	1	2026-04-01 10:00:00	10	25.00	Россия	Москва
2	1	Иван	Иванов	2	2026-04-02 12:00:00	5	15.00	Россия	Регионы
3	2	Петр	Петров	3	2026-04-03 18:30:00	3	30.00	Германия	Европа

Представление 2. Было создано представление, которое выводит самую популярную зону звонков за истекший год:

```
1 CREATE OR REPLACE VIEW phone_provider.popular_call_zone_year AS
2 SELECT
3     z.zone_id,
4     z.country,
5     z.region,
6     COUNT(c.call_id) AS call_count
7 FROM phone_provider.call c
8 JOIN phone_provider.call_zone z ON c.zone_id = z.zone_id
9 WHERE c.call_time >= CURRENT_DATE - INTERVAL '1 year'
10 GROUP BY z.zone_id, z.country, z.region
11 ORDER BY call_count DESC;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 47 msec.

Просмотр содержимого представления popular_call_zone_year:

```
1 SELECT * FROM phone_provider.popular_call_zone_year
2
```

	zone_id integer	country character varying (50)	region character varying (50)	call_count bigint
1	2	Россия	Регионы	1
2	1	Россия	Москва	1
3	3	Германия	Европа	1

4.3 Запросы на модификацию данных

Запрос 1 (INSERT с подзапросом). Добавить услугу всем абонентам из определенного города:

```
1  INSERT INTO phone_provider.service_connection (abonent_id, service_id, start_date)
2  SELECT
3      a.abonent_id,
4      3,
5      CURRENT_DATE
6  FROM phone_provider.abonent a
7  JOIN phone_provider.address ad ON a.address_id = ad.address_id
8  WHERE ad.city = 'Казань';
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 95 msec.

Теперь сделаем SELECT-запрос для проверки вставки в таблицу с подключенными услугами:

```
1  SELECT
2      a.abonent_id,
3      a.first_name,
4      a.last_name,
5      ad.city,
6      sc.service_id,
7      sc.start_date,
8      sc.end_date
9  FROM phone_provider.abonent a
10 JOIN phone_provider.address ad
11     ON a.address_id = ad.address_id
12 LEFT JOIN phone_provider.service_connection sc
13     ON a.abonent_id = sc.abonent_id
14 ORDER BY a.abonent_id;
```

Data Output Messages Notifications

	abonent_id integer	first_name character varying (50)	last_name character varying (50)	city character varying (30)	service_id integer	start_date date	end_date date
1	1	Иван	Иванов	Москва	1	2025-01-10	[null]
2	1	Иван	Иванов	Москва	2	2025-01-15	[null]
3	2	Петр	Петров	Санкт-Петербург	1	2025-02-10	[null]
4	3	Анна	Сидорова	Казань	3	2026-04-15	[null]

И видим, что услуга с id=3 подключена только у абонента из города Москва.

Запрос 2 (UPDATE с подзапросом). Обновление баланса абонентов на основе суммарной стоимости совершённых звонков.

Просмотрим абонентов и их баланс до UPDATE-запроса:

```
1  SELECT
2      a.abonent_id,
3      a.first_name,
4      a.last_name,
5      a.balance
6  FROM phone_provider.abonent a
```

	abonent_id [PK] integer	first_name character varying (50)	last_name character varying (50)	balance numeric (10,2)
1	1	Иван	Иванов	500.00
2	2	Петр	Петров	150.00
3	3	Анна	Сидорова	-50.00

Теперь выполним UPDATE-запрос:

```
1  UPDATE phone_provider.abonent a
2  SET balance = balance - (
3      SELECT COALESCE(SUM(c.total_cost), 0)
4      FROM phone_provider.call c
5      WHERE c.abonent_id = a.abonent_id
6  );
```

Data Output	Messages	Notifications
UPDATE 3		
Query returned successfully in 58 msec.		

И теперь опять выведем баланс абонентов с помощью SELECT:

```
1 SELECT
2     a.abonent_id,
3     a.first_name,
4     a.last_name,
5     a.balance
6 FROM phone_provider.abonent a
```

	abonent_id [PK] integer	first_name character varying (50)	last_name character varying (50)	balance numeric (10,2)
1	1	Иван	Иванов	460.00
2	2	Петр	Петров	120.00
3	3	Анна	Сидорова	-50.00

Видим, что баланс абонентов изменился в зависимости от стоимости звонков, которые они совершили.

Запрос 3 (DELETE с подзапросом). Удаление записей о завершенных подключениях услуг, связанных с неактивными сервисами.

Проверим таблицу подключений услуг до выполнения DELETE-запроса:

```
1 SELECT
2     sc.id,
3     sc.abonent_id,
4     sc.service_id,
5     sc.start_date,
6     sc.end_date,
7     s.status AS service_status
8 FROM phone_provider.service_connection sc
9 JOIN phone_provider.service s
10    ON sc.service_id = s.service_id
```

	id integer	abonent_id integer	service_id integer	start_date date	end_date date	service_status character varying (20)
1	1	1	1	2025-01-10	[null]	active
2	2	1	2	2025-01-15	[null]	active
3	3	2	1	2025-02-10	[null]	active
4	4	3	3	2026-04-15	[null]	inactive

Видим, что в 4 строке есть подключение с неактивной услугой.
Выполним DELETE-запрос:

```
1 DELETE FROM phone_provider.service_connection sc
2 WHERE sc.service_id IN (
3     SELECT s.service_id
4     FROM phone_provider.service s
5     WHERE s.status = 'inactive'
6 )
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 43 msec.

И теперь видим, что такой записи больше нет:

```
1 SELECT
2     sc.id,
3     sc.abonent_id,
4     sc.service_id,
5     sc.start_date,
6     sc.end_date,
7     s.status AS service_status
8 FROM phone_provider.service_connection sc
9 JOIN phone_provider.service s
10 ON sc.service_id = s.service_id
```

Data Output Messages Notifications

Showing rows: 1 to 3

	id integer	abonent_id integer	service_id integer	start_date date	end_date date	service_status character varying (20)
1	1	1	1	2025-01-10	[null]	active
2	2	1	2	2025-01-15	[null]	active
3	3	2	1	2025-02-10	[null]	active

4.4 Создание индексов и сравнение времени выполнения запросов

В таблицу абонентов было добавлено 2000+ записей, чтобы заметить разницу во времени выполнения запроса с помощью индекса и без него. В качестве тестового запроса сделаем выборку полей с балансом абонентов, где значения < 0 , то есть у абонентов имеется задолженность:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM phone_provider.abonent
4 WHERE balance < 0;
```

Data Output Messages Notifications

Showing rows: 1 to 5

	QUERY PLAN	
	text	
1	Seq Scan on abonent (cost=0.00..49.14 rows=989 width=61) (actual time=0.006..0.294 rows=991 loop...	
2	Filter: (balance < '0'::numeric)	
3	Rows Removed by Filter: 1020	
4	Planning Time: 0.399 ms	
5	Execution Time: 0.325 ms	

Благодаря использованию команды EXPLAIN ANALYZE, видим что: Результаты без использования индекса:

- Planning Time: 0.399 ms
- Execution Time: 0.325 ms

Далее был создан индекс `idx_abonent_balance` для поля `balance` из таблицы `abonent`:

```
1 CREATE INDEX idx_abonent_balance
2 ON phone_provider.abonent(balance);
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 46 msec.

Теперь еще раз выполняем SELECT-запрос:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM phone_provider.abonent
4 WHERE balance < 0;
```

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN	
1	Seq Scan on abonent (cost=0.00..49.14 rows=989 width=61) (actual time=0.011..0.279 rows=991 loop...	
2	Filter: (balance < '0'::numeric)	
3	Rows Removed by Filter: 1020	
4	Planning Time: 0.088 ms	
5	Execution Time: 0.312 ms	

И видим что с использованием индекса:

- Planning Time: 0.088 ms
- Execution Time: 0.312 ms

Из полученных данных видно, что время выполнения запроса (Execution Time) практически не изменилось. Это объясняется тем, что объем данных в таблице относительно небольшой.

Но наблюдается можно заметить уменьшение времени планирования запроса (Planning Time) после создания индекса. Это связано с тем, что планировщик PostgreSQL получил дополнительную возможность выбора более эффективного плана выполнения запроса благодаря наличию индекса.

5 Вывод

В ходе выполнения лабораторной работы были созданы запросы на выборку данных, построены представления, составлены запросы на модификацию данных с использованием подзапросов, изучено использование команды EXPLAIN ANALYZE, а также исследовано влияние индексов на время выполнения запросов.

В результате выполнения работы были приобретены практические навыки:

- составления запросов на выборку данных к базе данных PostgreSQL;
- создания и использования представлений;
- применения подзапросов в запросах на модификацию данных;
- анализа истории запросов и графического плана выполнения;
- создания простых и составных индексов;
- сравнительного анализа времени выполнения запросов без индексов и с индексами.